

Lista de tareas

Una explicación corta de cómo está hecha la aplicación y cómo se conectan sus partes.

En una frase: la app permite escribir tareas, moverlas entre pendientes y realizadas, y borrar las ya terminadas.

1. ¿Qué pasa cuando usamos la app?

La persona escribe una tarea. La app revisa que tenga entre 3 y 60 caracteres y, si es válida, la agrega a la lista de pendientes. Al marcar la casilla, la tarea pasa a realizadas. Desde allí se pueden eliminar todas juntas.

2. Archivos principales

Archivo	¿Para qué sirve?
App.jsx	Muestra la pantalla y coordina el formulario y las listas.
taskUtils.js	Contiene funciones pequeñas para validar, crear y actualizar tareas.
main.jsx	Pone el componente principal dentro de la página web.
App.css y index.css	Definen el aspecto, los colores y el diseño adaptable.
package.json	Indica las dependencias y los comandos del proyecto.

3. ¿Qué datos guarda cada tarea?

```
{
  id: "identificador-unico",
  descripcion: "Comprar pan",
  hecho: false
}
```

id identifica una tarea. **descripcion** contiene el texto escrito. **hecho** vale **false** si está pendiente y **true** si está realizada.

4. App.jsx: el centro de la aplicación

Este archivo contiene el componente **App**. Su estado es la información que React recuerda mientras la página está abierta: el texto del formulario y la lista de tareas.

```
const [description, setDescription] = useState("");
const [tasks, setTasks] = useState([]);
```

useState entrega un valor y una función para cambiarlo. Al cambiar el estado, React vuelve a dibujar la parte de la pantalla que lo necesita.

Paso 1: revisar el texto

El botón se habilita cuando la descripción, sin espacios sobrantes, tiene entre 3 y 60 caracteres. El campo también limita la escritura a 60.

```
const canSubmit = isValidDescription(description);
```

Paso 2: agregar una tarea

El formulario responde al botón y a Enter. Si el texto es válido, crea una tarea, la agrega al estado y deja el campo vacío.

```
setTasks((current) => [...current, createTask(description)]);
setDescription("");
```

Paso 3: separar pendientes y realizadas

Las dos tablas no guardan listas distintas: se calculan a partir de **tasks**. Así, cambiar **hecho** mueve automáticamente la tarea de una tabla a la otra.

```
const pendingTasks = tasks.filter((task) => !task.hecho);
const completedTasks = tasks.filter((task) => task.hecho);
```

Paso 4: cambiar o eliminar tareas

Cada checkbox llama a **handleToggle**, que actualiza solamente la tarea elegida. El botón “Eliminar realizadas” conserva las que siguen pendientes.

```
setTasks((current) => toggleTask(current, id));
setTasks((current) => removeCompleted(current));
```

Importante: la lista vive en la memoria de la página. Si se recarga el navegador, las tareas se borran; no se usa una base de datos ni almacenamiento local.

5. taskUtils.js: reglas pequeñas

Función	En palabras simples
<code>isValidDescription</code>	Comprueba que el texto tenga de 3 a 60 caracteres.
<code>createTask</code>	Crea el objeto con un ID único y hecho: false .
<code>toggleTask</code>	Cambia el valor de hecho de una tarea.
<code>removeCompleted</code>	Devuelve solo las tareas que aún están pendientes.

Ejemplo: crear una tarea

```
export function createTask(description) {
  return {
    id: crypto.randomUUID(),
    descripcion: description.trim(),
    hecho: false,
  };
}
```

`trim()` quita espacios al principio y al final. `crypto.randomUUID()` genera un identificador único para usarlo también como clave de la fila en React.

6. ¿Para qué sirven los archivos CSS?

`index.css` define estilos generales, como la letra, el fondo y el foco visible al usar el teclado. `App.css` da formato al formulario y las tablas. Una regla para pantallas angostas apila el campo y el botón para que sea más fácil usar la app desde un teléfono.

7. Ejecutar el proyecto

Abre una terminal en la carpeta **3- Tareas** y usa estos comandos:

```
npm install
npm run dev
```

Vite mostrará una dirección local para abrir en el navegador. Para verificar las funciones y preparar una versión de producción:

```
npm test
npm run build
```

Resumen: `App.jsx` se ocupa de la pantalla y su estado; `taskUtils.js` aplica las reglas de las tareas; los estilos hacen que la pantalla sea clara y adaptable.